DATA STRUCTURE AND ALGORITHM

Programming Practice Session

14/11/2025

4th Session

- List ADT
- Singly Linked List
- Doubly Linked List
- Circular Linked List
- Binary Search Tree

Revision

List Abstract Data Type:

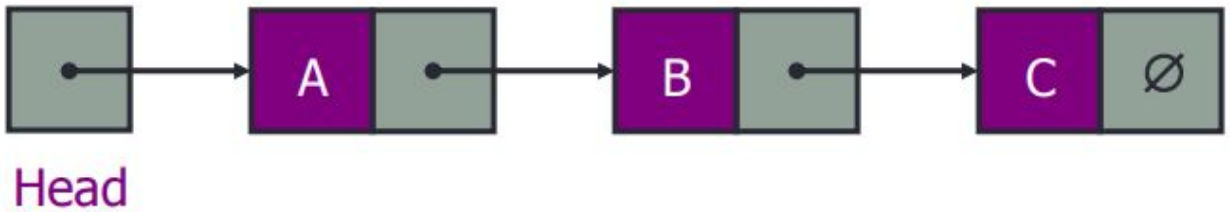
- Ordered, same type, index access.
- Implementation: array-based, linked list.
- ……allocate the array.
- Shift elements when inserting or deleting.

```
class ArrayList {
private:
#define MAX_SIZE  100
  int  elements[MAX_SIZE];
  int noOfElements;
```

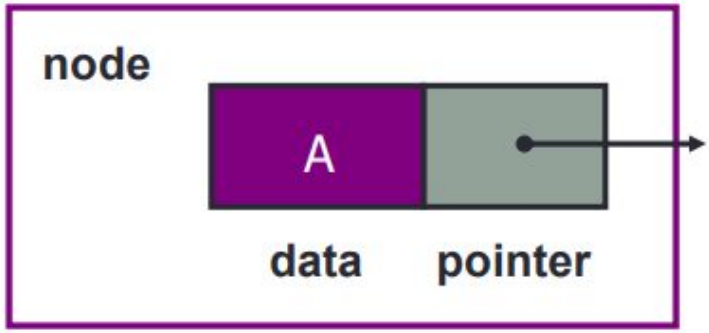
```
public:
  ArrayList (); // Constructor
  ~ArrayList ();// Destructor

  void Insert(int e, int pos);
  void Delete(int pos);
  int Find(int e);
  bool IsEmpty();
  int First();
  int Last();
  int Kth(int pos);
  int Length();
};
```

```
// Constructor:
// Make empty list
ArrayList::ArrayList(){
    noOfElements = 0;
} // end-ArrayList
```

Revision



- A *linked list* is a series of connected *nodes*
- Each node contains at least
  - A piece of data (any type)
  - Pointer to the next node in the list
- *Head*: pointer to the first node
- The last node points to `NULL`



Revision

We use two classes: Node and List

Declare Node class for the nodes

- data: double-type data in this example
- next: a pointer to the next node in the list

```
class Node {
public:
      double      data;       // data
      Node*       next;       // pointer to next
};
```

Declare List, which contains

- head: a pointer to the first node in the list.
  Since the list is empty initially, head is set to NULL
- Operations on List

```
class List {
public:
      List(void) { head = NULL; }       // constructor
      ~List(void);                      // destructor

      bool IsEmpty() { return head == NULL; }
      Node* InsertNode(int index, double x);
      int FindNode(double x);
      int DeleteNode(double x);
      void DisplayList(void);
private:
      Node* head;
};
```

Linked list:

- Size?: …
- When inserting/deleting: …
- How to access? => O(...)
- Take up more …. for….

Revision

Binary tree: data structure in ….. Form, each node have maximum …. Child.

Root: no parent node.

A Node contains: data, ….., ……

Height of tree: number of …… edges from root to leaf

Level:

+ Root: level 0
+ Root's children: …
+ ……

A Subtree starts from any node.

Tree Traversal: …, …, …

Revision

Binary search tree: a special form of binary tree:

+ Left subtree < Node
+ Right subtree > Node
+ Apply recursively to every node

=> makes searching faster

Search:

+ Key == node-> data: return node
+ Key < node->data: go left
+ Key > node-> data: go right

Complexity: O(....) on avg.

Assignment 4

Create content in main function to test for each problem. Do not use built-in functions and remember to comment code.

1 (In class) Implementing array-based ListADT, Linked List, Binary Search Tree with the functions in previous slides and theory slide.

2. Implement List ADT using dynamic array with functions: insert(pos, x) — insert element x at position pos; remove(pos) — remove element at position pos; get(pos) — get value at position pos; size() — return number of elements in list; display() — print entire list.

3. Merge two sorted singly linked lists into one sorted list.

4. Implement doubly linked list with functions as in singly linked list.

5. Reverse a doubly linked list.

Assignment 4

6. Implement singly circular linked list with functions as in singly linked list.

7. Implement doubly circular linked list with functions as in singly linked list at both ends.

8. Implement BST with:

- Insert, delete, search operations.
- Find minimum, maximum, successor, and predecessor in BST.
- Print elements in inorder, preorder, and postorder traversals.
- Check if a given binary tree is a BST or not.
- Find lowest common ancestor (LCA) of two nodes in BST.
- Convert sorted array to balanced BST.
- Find kth smallest/largest element in BST.